Program Code:

```
#include <iostream>
using namespace std;

int partition(int arr[],int low,int high)
{
int pivot=arr[high];
int i=(low-1);

for(int j=low;j<=high;j++)
{
  if(arr[j]<pivot)
  {
  i++;
  swap(arr[i],arr[j]);
  }
}
swap(arr[i+1],arr[high]);
return (i+1);
}
void quickSort(int arr[],int low,int high)
{
if(low<high)
{

  int pi=partition(arr,low,high);
  quickSort(arr,low,pi-1);
  quickSort(arr,pi+1,high);
}
}
int main() {
int n;
cout<<"Enter size of array: ";
cin>>n;
cout<<"Enter the array:"<<endl;
int arr[n];
for(int i=0;i<n;i++)
    cin>>arr[i];
cout<<"Original Array: ";
for(int i=0;i<n;i++)
    cout<<arr[i]<<" ";
quickSort(arr,0,n-1);
cout<<"\nSorted Array: ";
for(int i=0;i<n;i++)
{
  cout<<arr[i]<<" ";
}
return 0;
}
```


Program Code:

```
#include <iostream>
using namespace std;

int partition(int arr[],int low,int high)
{
int pivot=arr[high];
int i=(low-1);

for(int j=low;j<=high;j++)
{
  if(arr[j]<pivot)
  {
  i++;
  swap(arr[i],arr[j]);
  }
}
swap(arr[i+1],arr[high]);
return (i+1);
}
void quickSort(int arr[],int low,int high)
{
if(low<high)
{

  int pi=partition(arr,low,high);
  quickSort(arr,low,pi-1);
  quickSort(arr,pi+1,high);
}
}
int main() {
int n;
cout<<"Enter size of array: ";
cin>>n;
cout<<"Enter the array:"<<endl;
int arr[n];
for(int i=0;i<n;i++)
    cin>>arr[i];
cout<<"Original Array: ";
for(int i=0;i<n;i++)
    cout<<arr[i]<<" ";
quickSort(arr,0,n-1);
cout<<"\nSorted Array: ";
for(int i=0;i<n;i++)
{
  cout<<arr[i]<<" ";
}
return 0;
}
```

Output

```
/tmp/6wJm69RuLK.o
Enter size of array: 8
Enter the array:
12
4
5
6
7
3
1
15
Original Array: 12 4 5 6 7 3 1 15
Sorted Array: 1 3 4 5 6 7 12 15
```

Output

```
/tmp/6wJm69RuLK.o
Enter size of array: 8
Enter the array:
12
4
5
6
7
3
1
15
Original Array: 12 4 5 6 7 3 1 15
Sorted Array: 1 3 4 5 6 7 12 15
```

Output

```
/tmp/13V68q2Lmt.o
Enter Size of Array: 8
Enter the Array: 81 89 9 11 14 76 54 22
Given array is
81 89 9 11 14 76 54 22
Sorted array is
9 11 14 22 54 76 81 89


=== Code Execution Successful ===
```

Program Code:

```
#include <stdio.h>
void swap(int* a, int* b)
{
   int temp = *a;
   *a = *b;
   *b = temp;
}
void heapify(int arr[], int N, int i)
{
   int largest = i;
   int left = 2 * i + 1; // left = 2*i + 1
   int right = 2 * i + 2;   // right = 2*i + 2
   if (left < N && arr[left] > arr[largest])
      largest = left;
   if (right < N && arr[right] > arr[largest])
      largest = right;
   if (largest != i) {
      swap(&arr[i], &arr[largest]);
      heapify(arr, N, largest);
   }
}
void heapSort(int arr[], int N)
{
   for (int i = N / 2 - 1; i >= 0; i--)
      heapify(arr, N, i);
   for (int i = N - 1; i >= 0; i--) {
      swap(&arr[0], &arr[i]);
      heapify(arr, i, 0);
   }
}
void printArray(int arr[], int N)
{
   for (int i = 0; i < N; i++)
      printf("%d ", arr[i]);
   printf("\n");
}
int main()
{
   int n;
   printf("Enter Size of Array: ");
   scanf("%d",&n);
   printf("Enter the Array: ");
   int arr[n];
   for(int i=0;i<n;i++)
   scanf("%d",&arr[i]);
   printf("Given array is \n");
   printArray(arr, n);
   heapSort(arr, n);
   printf("Sorted array is\n");
   printArray(arr, n);
}
```

Program Code:

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_VERTICES 100
struct Node {
	int data;
	struct Node* next;
};
struct Node* createNode(int data) {
	struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
	newNode->data = data;
	newNode->next = NULL;
	return newNode;
}
void addEdge(struct Node* adjList[], int u, int v) {
	struct Node* newNode = createNode(v);
	newNode->next = adjList[u];
	adjList[u] = newNode;
}
void bfs(struct Node* adjList[], int vertices, int startNode, int visited[]) {
	int queue[MAX_VERTICES];
	int front = 0, rear = 0;
	visited[startNode] = 1;
	queue[rear++] = startNode;
	while (front != rear) {
		int currentNode = queue[front++];
		printf("%d ", currentNode);
		struct Node* temp = adjList[currentNode];
		while (temp != NULL) {
			int neighbor = temp->data;
			if (!visited[neighbor]) {
				visited[neighbor] = 1;
				queue[rear++] = neighbor;
			}
			temp = temp->next;
		}}}
int main() {
	int vertices = 5;
	struct Node* adjList[vertices];
	for (int i = 0; i < vertices; ++i)
		adjList[i] = NULL;
	addEdge(adjList, 0, 1);
	addEdge(adjList, 0, 2);
	addEdge(adjList, 1, 3);
	addEdge(adjList, 1, 4);
	addEdge(adjList, 2, 4);
	int visited[vertices];
	for (int i = 0; i < vertices; ++i)
		visited[i] = 0;
	printf("Breadth First Traversal starting from vertex 0: ");
	bfs(adjList, vertices, 0, visited);
  return 0;   }
```

```
Breadth First Traversal starting from vertex 0: 0 2 1 4 3

...Program finished with exit code 0
Press ENTER to exit console.
```

Output

```
java -cp /tmp/jgDrYmwmTb/OddEvenMergeSorter
1 2 3 6 9 10 11 17
=== Code Execution Successful ===
```

Output

```
/tmp/EYyPf5hu46.o
Vertex       Distance from Source
A                 0
B                 2
C                 5
D                 4
E                 8
F                 10
G                 8


=== Code Execution Successful ===
```

Program Code:

```
#include <stdio.h>
#include <stdlib.h>
struct node {
  int vertex;
  struct node* next;
};
struct node* createNode(int v);
struct Graph {
  int numVertices;
  int* visited;
  struct node** adjLists;
};
void DFS(struct Graph* graph, int vertex) {
  struct node* adjList = graph->adjLists[vertex];
  struct node* temp = adjList;
  graph->visited[vertex] = 1;
  printf("%d ", vertex);
  while (temp != NULL) {
    int connectedVertex = temp->vertex;
    if (graph->visited[connectedVertex] == 0) {
      DFS(graph, connectedVertex);
    }
    temp = temp->next;
  }
}
struct node* createNode(int v) {
  struct node* newNode = malloc(sizeof(struct node));
  newNode->vertex = v;
  newNode->next = NULL;
  return newNode;
}
struct Graph* createGraph(int vertices) {
  struct Graph* graph = malloc(sizeof(struct Graph));
  graph->numVertices = vertices;
  graph->adjLists = malloc(vertices * sizeof(struct node*));
  graph->visited = malloc(vertices * sizeof(int));
  int i;
  for (i = 0; i < vertices; i++) {
    graph->adjLists[i] = NULL;
    graph->visited[i] = 0;
  }
  return graph;
}
void addEdge(struct Graph* graph, int src, int dest) {
  struct node* newNode = createNode(dest);
  newNode->next = graph->adjLists[src];
  graph->adjLists[src] = newNode;
  newNode = createNode(src);
  newNode->next = graph->adjLists[dest];
  graph->adjLists[dest] = newNode;
}
```

```
// Print the graph
void printGraph(struct Graph* graph) {
  int v;
  for (v = 0; v < graph->numVertices; v++) {
    struct node* temp = graph->adjLists[v];
    printf("\n Adjacency list of vertex %d\n ", v);
    while (temp) {
      printf("%d -> ", temp->vertex);
      temp = temp->next;
    }
    printf("\n");
  }
}
int main() {
  struct Graph* graph = createGraph(4);
  addEdge(graph, 0, 1);
  addEdge(graph, 0, 2);
  addEdge(graph, 1, 2);
  addEdge(graph, 2, 3);
  printGraph(graph);
  printf("\n");
  DFS(graph, 0);
  return 0;
}
```

Output:

Output

```
/tmp/KyssJcEx4g.o

 Adjacency list of vertex 0
 2 -> 1 ->

 Adjacency list of vertex 1
 2 -> 0 ->

 Adjacency list of vertex 2
 3 -> 1 -> 0 ->

 Adjacency list of vertex 3
 2 ->

0 2 3 1

=== Code Execution Successful ===
```

Program Code:

```
#include<stdio.h>
#include <stdlib.h>
void makeSet(int parent[], int rank[], int n)
{
	for (int i = 0; i < n; i++) {
		parent[i] = i;
		rank[i] = 0;
	}
}
int findParent(int parent[], int component)
{
	if (parent[component] == component)
		return component;

	return parent[component]
		= findParent(parent, parent[component]);
}
int comparator(const void* p1, const void* p2)
{
	const int(*x)[3] = p1;
	const int(*y)[3] = p2;
	return (*x)[2] - (*y)[2];
}
void unionSet(int u, int v, int parent[], int rank[], int n)
{
	u = findParent(parent, u);
	v = findParent(parent, v);

	if (rank[u] < rank[v]) {
		parent[u] = v;
	}
	else if (rank[u] > rank[v]) {
		parent[v] = u;
	}
	else {
		parent[v] = u;
		rank[u]++;
	}
}
void kruskalAlgo(int n, int edge[n][3])
{
	// First we sort the edge array in ascending order
	qsort(edge, n, sizeof(edge[0]), comparator);
	int parent[n];
	int rank[n];
	makeSet(parent, rank, n);
	int minCost = 0;
	printf( "Following are the edges in the constructed MST\n");
	for (int i = 0; i < n; i++) {
		int v1 = findParent(parent, edge[i][0]);
		int v2 = findParent(parent, edge[i][1]);
		int wt = edge[i][2];

		if (v1 != v2) {
			unionSet(v1, v2, parent, rank, n);
			minCost += wt;
			printf("%d -- %d == %d\n", edge[i][0],
				edge[i][1], wt);
		}
	}
	printf("Minimum Cost Spanning Tree: %d\n", minCost);
}
int main()
{
	int edge[10][3] = { { 0, 1, 7 },
				{ 0, 2, 8 },
				{ 1, 2, 3 },
				{ 1, 3, 6 },
				{ 1, 5, 5 },
				{ 2, 3, 4 },
				{ 2, 5, 3 },
				{ 3, 4, 5 },
				{ 3, 5, 2 },
				{ 4, 5, 2 } };
	kruskalAlgo(10, edge);
	return 0;
}
```

Output

```
/tmp/RkI4K4FFPk.o
Following are the edges in the constructed MST
3 -- 5 == 2
4 -- 5 == 2
1 -- 2 == 3
2 -- 5 == 3
0 -- 1 == 7
Minimum Cost Spanning Tree: 17


=== Code Execution Successful ===
```

Program Code:

```
#include <limits.h>
#include <stdbool.h>
#include <stdio.h>
// Number of vertices in the graph
#define V 7
int minDistance(int dist[], bool sptSet[]) {
	// Initialize min value
	int min = INT_MAX, min_index;
	for (int v = 0; v < V; v++)
		if (sptSet[v] == false && dist[v] <= min)
			min = dist[v], min_index = v;
	return min_index;
}
void printSolution(int dist[]) {
	printf("Vertex \t\t Distance from Source\n");
	for (int i = 0; i < V; i++)
		printf("%c \t\t\t\t %d\n", i+65, dist[i]);
}
void dijkstra(int graph[V][V], int src)
{
	int dist[V];
	bool sptSet[V];
   for (int i = 0; i < V; i++)
		dist[i] = INT_MAX, sptSet[i] = false;
	dist[src] = 0;
   for (int count = 0; count < V - 1; count++) {
		int u = minDistance(dist, sptSet);
		sptSet[u] = true;
		for (int v = 0; v < V; v++)
			if (!sptSet[v] && graph[u][v]

				&& dist[u] != INT_MAX
				&& dist[u] + graph[u][v] < dist[v])
				dist[v] = dist[u] + graph[u][v];
	}
   printSolution(dist);
}
int main()
{
	int graph[V][V] = {		{ 0, 2, 6, 0, 0, 0, 0},
						{ 2, 0, 0, 2, 0, 0, 0},
						{ 6, 0, 0, 1, 0, 0, 3},
						{ 0, 2, 1, 0, 4, 0, 5},
						{ 0, 0, 0, 4, 0, 2, 0},
						{ 0, 0, 0, 0, 2, 0, 6},
						{ 0, 0, 3, 5, 0, 6, 0} };

	dijkstra(graph, 0);

	return 0;
}
```

Program Code:

```
public class OddEvenMergeSorter
{
   private int[] a;
   public void sort(int[] a)
   {
      this.a=a;
      oddEvenMergeSort(0, a.length);
      display();
   }
   private void oddEvenMergeSort(int lo, int n)
   {
      if (n>1)
      {
         int m=n/2;
         oddEvenMergeSort(lo, m);
         oddEvenMergeSort(lo+m, m);
         oddEvenMerge(lo, n, 1);
      }
   }
   private void oddEvenMerge(int lo, int n, int r)
   {
      int m=r*2;
      if (m<n)
      {
         oddEvenMerge(lo, n, m);     // even subsequence
         oddEvenMerge(lo+r, n, m);   // odd subsequence
         for (int i=lo+r; i+r<lo+n; i+=m)
            compare(i, i+r);
      }
      else
         compare(lo, lo+r);
   }
   private void compare(int i, int j)
   {
      if (a[i]>a[j])
         exchange(i, j);
   }
   private void exchange(int i, int j){
      int t=a[i];
      a[i]=a[j];
      a[j]=t;
   }
   private void display(){
      for(int i=0;i<a.length;i++)
         System.out.print(a[i]+" ");
   }
   public static void main(String[] args){
      OddEvenMergeSorter s=new OddEvenMergeSorter();
      int ar[]={9,3,17,11,6,2,1,10};
      s.sort(ar);
   }}
```